AdocNet v1.0.0-beta.4 — Feature Showcase

Syntax Highlighting

Beta.4 adds server-side syntax highlighting for source blocks. Seven languages are supported with 9 token categories.

C# Example

csharp

```csharp
using System.Text;

namespace AdocNet.Highlighting;

/// <summary>Tokenizes source code for syntax highlighting.</summary>
public static class SyntaxTokenizer
{
    public static List<SyntaxToken> Tokenize(string source, string? language)
    {
        if (string.IsNullOrEmpty(source))
            return [new SyntaxToken(TokenKind.Plain, source ?? "")];

        #if DEBUG
        Console.WriteLine($"Tokenizing {source.Length} chars as {language}");
        #endif

        return TokenizeWithRules(source, rules);
    }

    [Obsolete("Use Tokenize instead")]
    public static bool IsSupported(string lang) => true;
}
```

Python Example

python

```python
import json
from pathlib import Path

class DocumentProcessor:
    """Processes AsciiDoc documents with syntax highlighting."""

    def __init__(self, config_path: str = "config.json"):
        self.config = self._load_config(config_path)
        self._cache: dict[str, list] = {}

    def process(self, source: str, language: str = "python") -> list:
        # Check cache first
        if source in self._cache:
            return self._cache[source]

        tokens = self._tokenize(source, language)
        self._cache[source] = tokens
        return tokens

    @staticmethod
    def _load_config(path: str) -> dict:
```

```
        """Load configuration from JSON file."""
        with open(path, 'r') as f:
            return json.load(f)
```

JavaScript Example

javascript

```javascript
const express = require('express');
const app = express();

// Middleware for JSON parsing
app.use(express.json());

app.get('/api/documents/:id', async (req, res) => {
    const { id } = req.params;
    const limit = parseInt(req.query.limit) || 10;

    try {
        const doc = await Document.findById(id);
        if (!doc) {
            return res.status(404).json({ error: "Not found" });
        }
        res.json({ data: doc, count: limit });
    } catch (err) {
        console.error(`Failed to fetch document ${id}:`, err);
        res.status(500).json({ error: "Internal error" });
    }
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

SQL Example

sql

```sql
SELECT u.username, COUNT(d.id) AS document_count,
       MAX(d.created_at) AS last_document
FROM users u
LEFT JOIN documents d ON d.author_id = u.id
WHERE u.active = 1
  AND d.created_at >= '2024-01-01'
GROUP BY u.username
HAVING COUNT(d.id) > 5
ORDER BY document_count DESC
LIMIT 20;
```

JSON Configuration

json

```json
{
    "rendering": {
        "syntax_highlighting": true,
        "theme": "default",
        "languages": ["csharp", "python", "javascript", "sql", "json", "xml"],
        "token_count": 9,
        "quality_target": 0.8
    },
    "pdf": {
        "page_size": "A4",
        "margins": [72, 72, 72, 72],
        "font_size": 11.0,
        "enable_hyphenation": false
```

```
    }
}
```

XML/HTML Example

xml
```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Configuration for the document renderer -->
<renderer version="1.0">
    <output format="pdf">
        <page width="595" height="842" />
        <margins top="72" bottom="72" left="72" right="72" />
    </output>
    <fonts>
        <font name="body" path="/fonts/arial.ttf" />
        <font name="mono" path="/fonts/consola.ttf" />
    </fonts>
</renderer>
```

Typography Improvements

Hyphenation

This paragraph demonstrates the Liang/Knuth hyphenation algorithm using standard TeX US English patterns. The internationalization and standardization of documentation requires comprehensive consideration of typographical conventions, including proper hyphenation of polysyllabic words. Without hyphenation, justified text can suffer from unsightly inter-word spacing, particularly in narrow columns. The implementation handles compound words, technical terminology, and common English vocabulary with approximately eighty percent accuracy, which is sufficient for professional documentation.

Paragraph Spacing

This is the first paragraph demonstrating configurable paragraph spacing. The spacing before and after paragraphs is now independently controllable via `ParagraphSpacingBefore` and `ParagraphSpacingAfter` properties.

This second paragraph follows with the configured spacing. Section spacing is also independently configurable via the `SectionSpacing` property.

PDF Styling

Heading Colors

The section headings in this document are rendered with a custom color using the `HeadingColor` property of `PdfRenderOptions`.

Style Presets

Two convenience presets are available:

- **Compact**: `FontSize=10, LineSpacing=1.25, ParagraphSpacingAfter=6, MarginTop=54, MarginBottom=54` - **Presentation**: `TitleFontSize=30, FontSize=14, LineSpacing=1.5, HeadingColor=(0, 0, 0.6)`

Tables

Feature	Status	Renderer
Syntax highlighting	7 languages	HTML + PDF
Hyphenation	English (US)	PDF only
Custom themes	4 built-in	HTML
Style presets	Compact, Presentation	PDF
Heading colors	Any RGB	PDF

Admonitions

NOTE:
Syntax highlighting defaults to disabled for backward compatibility. Enable it explicitly with `EnableSyntaxHighlighting = true` (HTML) or `SyntaxColors = SyntaxColorScheme.Default` (PDF).

TIP:
Use `PdfRenderOptions.Compact` for dense technical documentation and `PdfRenderOptions.Presentation` for slide-like output.

WARNING:
The syntax tokenizer targets 80% accuracy. Nested string interpolation, regex literals, and some multi-line constructs may not highlight correctly.

IMPORTANT:
All PDF output remains fully deterministic. Identical input always produces byte-identical output.

Conclusion

Beta.4 transforms AdocNet’s rendering from "correct and readable" to "professional and customizable" with:

- Server-side syntax highlighting for 7 languages - Liang/Knuth hyphenation with TeX patterns - 4 built-in HTML themes with syntax highlighting CSS - Configurable PDF styling (colors, spacing, presets) - 84 new tests ensuring quality and backward compatibility